



Gestão da Configuração e Versionamento

CÁSSIO CAPUCHO PEÇANHA

O caminho feliz

- Adicionar / Salvar / Enviar ...
- *"Professor, apaguei a pasta do projeto sem querer!"*
- *"Mande a senha do banco de dados pro GitHub!"*
- *É (quase) impossível perder código no Git se usarem os commits corretamente, eles param de ter medo de programar.*

"Onde" antes do "Como"

- Você precisa entender ONDE o erro está antes de escolher o comando para consertar.
- Vamos dividir a aula em 3 Níveis de Pânico:
 - **Pânico Nível 1 (O Rascunho):** Escrevi besteira no VS Code, mas não dei add nem commit.
 - **Pânico Nível 2 (O Palco):** Dei git add num arquivo que estava quebrado.
 - **Pânico Nível 3 (O Álbum):** Dei git commit numa alteração errada.

O Comando **git checkout**

- O Git Checkout é uma das ferramentas mais versáteis do Git, funcionando basicamente como o seu "meio de transporte" entre diferentes estados do seu projeto.
- Pense nele como uma forma de dizer ao Git: "Ei, quero ver como o código estava neste ponto específico agora".
- **1. Trocar de Branch**
- **2. Restaurar Arquivos (Descartar Mudanças)**
- **3. Explorar Commits Antigos**

Aposentando o ~~checkout~~

- Historicamente, usávamos o `git checkout` para tudo (mudar de branch e restaurar arquivos). Isso confundia muito os iniciantes.
- Nas versões recentes, o Git dividiu isso:
 - **`git switch`** (Só para trocar de ramificação/branch).
 - **`git restore`** (Só para restaurar arquivos).
- Foque no restore! É muito mais semântico e fácil para lembrar.

Pânico Nível 1

- **Erro 1:** "Deletei o arquivo main.c sem querer!" ou "Fiz um código horrível e quero voltar pro último save".
- **A Cura:**
- `git restore main.c`
- O Git vai no último commit e copia o arquivo perfeito de volta para a sua pasta, sobrescrevendo a besteira.

Pânico Nível 2

- **Erro 2:** "Dei git add . e mandei o arquivo .exe pro palco!".
- **A Cura:**
- `git restore --staged *.exe`
- Tira do palco e devolve pro camarim, como vimos na sua dúvida anterior.

Pânico Nível 3

- O reset mexe na linha do tempo
 - move o ponteiro HEAD para o passado.
- **A Rede de Segurança (--soft):** Volta no tempo, apaga a "foto", mas deixa o código modificado no Palco para você arrumar.
- **O Botão de Autodestruição (--hard):** Cuidado extremo! Volta no tempo, apaga a foto E joga seu código no lixo. Deixa a sua pasta de trabalho exatamente como era no passado.

Pânico Nível 3

- **Erro 3:** "Dei o git commit, mas percebi na mesma hora que esqueci de dar add num arquivo importante!" ou "Digitei a mensagem do commit cheia de erros de português!".
 - O seu código está ótimo, o problema foi apenas a forma como você salvou a foto. Você não quer perder o que digitou.
- A Cura (A Rede de Segurança):
- `git reset --soft HEAD~1`
- O Git volta 1 passo no tempo (HEAD~1). Ele rasga a última foto oficial do álbum, mas devolve todos os seus arquivos intactos para o Palco (Staging Area).
- Agora você pode adicionar o arquivo que faltava (`git add arquivo_esquecido.c`) e bater uma nova foto com a mensagem certa.

Pânico Nível 3

- **Erro 3:** "Tentei inventar uma função nova, quebrei o sistema inteiro, não sei mais como consertar e dei um commit nisso tudo por desespero!".
- O seu código está irremediavelmente quebrado. Você quer fingir que a última hora de trabalho simplesmente nunca existiu e voltar para o último "Save Point" seguro.
- A Cura (O Botão de Autodestruição): ``bash
- `git reset --hard HEAD~1`
- **Cuidado máximo!** O Git volta 1 passo no tempo, rasga a última foto do álbum E joga todo o seu código modificado no lixo.
- Os seus arquivos voltam a ser exatamente e perfeitamente idênticos ao que eram no commit anterior. Não tem "Ctrl+Z" depois disso.